Exploring multi-model orchestration for high-frequency trading platforms

Daniel–Ştefan Halasz
vu\dha242
Student no. 2804542
d.halasz@student.vu.nl

Vitor Andrade Fernández
vu\van204
Student no. 2798043
v.andrade.fernandez@student.vu.nl

Karina Kalicka–Molin
vu\qjy548
Student no. 2861685
k.b.kalicka-molin@student.vu.nl

***Abstract*—High-frequency trading is characterized by rapid regime shifts and non-linearity, challenging the effectiveness of single-model market prediction approaches. Deep learning models have shown promise in capturing the temporal limit order book features, but their performance can degrade during regime shifts. Therefore, no single model can consistently perform well across all market conditions, justifying the need for a meta-model prediction system that can dynamically select the best-performing model based on current market conditions. In this work, we explore the potential of dynamic model selection for high-frequency trading by implementing a machine learning meta-model orchestrator, evaluated on the FI-2010 dataset. In accordance with state-of-the-practice of high-frequency trading, we evaluate the trade-off between predictive performance and inference latency, considering the fact that a model's latency can impact its practical utility in real-world scenarios. We employ a diverse set of baseline models, and two meta-models: a logistic regression and a multi-layer perceptron, to predict the best-performing model for each time step. Our results show that the non-linear multi-layer perceptron orchestrator achieves competitive predictive performance compared to the best-performing baseline model, but fails to scale in terms of inference latency, being bottlenecked by the latency of the worst-performing baseline model.**

I. Introduction

PREDICTING market trends is a critical aspect of making informed trading decisions in the financial industry. Accurate predictions lead to more profitable investment strategies and increased returns. Symmetrically, inaccurate predictions carry significant risks, ranging from financial losses to missed opportunities. Machine learning (ML) techniques have been increasingly applied to the finance sector, aiming to increase the accuracy of market predictions. Kelly et al. [1] provide an overview of the shift towards data-driven architectures in financial modeling, highlighting the potential of ML to enhance predictive capabilities. In short, the finance industry has witnessed a significant paradigm shift in its approach to estimating market behavior, transitioning from traditional linear models to complex ML-based models that can capture non-linear interactions in financial data. With the newly perfected ML-based strategies, investors can execute trades at unprecedented speeds, capitalizing on even the smallest market movements; thus, high-frequency trading (HFT) has become a dominant force in the financial markets. As of late 2025, empirical evidence indicates that the footprint of HFT in the US financial markets reached approximately 73% of the total trading volume, while in the European markets, HFT accounts for around 50% of the trading volume [2]–[4]. Therefore, work towards improving the accuracy of HFT strategies, while reducing model inference is highly rewardable, as it can yield increased financial gains for investors and firms operating in the financial markets.

The adoption of ML-based strategies in trading poses significant challenges, particularly in the context of HFT (or in other words, short-term trading), where the market is unpredictable in its volatility, and highly dynamic in its behavior. In such conditions, a reoccurring problem is the lack of a generalizable model that can consistently outperform the market across different time periods and market conditions. For example, research shows that recurrent neural network (RNN)-based models excel in capturing complex, non-linear relationships in high-volatility market regimes, while linear models perform better during stable periods, because they are more likely to overfit on noise data, which is a characteristic of fast-moving markets [5], [6]. This phenomenon (often referred to as the *no free lunch* problem) highlights the need for adaptive strategies in HFT platforms. Current work [4] towards addressing the need of adaptability disregards the computational latency cost of the proposed solutions, which is a critical aspect in HFT platforms. If the latency of the prediction model exceeds the shelf-life of the predictive signal, the accuracy gain becomes economically irrelevant. A review of current literature reveals a gap in research that explicitly considers the trade-off between prediction accuracy and computational latency in adaptive market prediction strategies.

In this work, we propose a novel approach towards adaptive market prediction in HFT platforms, which is based on an orchestrator model that dynamically selects the most suitable prediction model based on current market conditions. The addition of an orchestrator introduces an additional layer of complexity, and thus, increased latency, which can potentially negate the benefits of improved prediction accuracy. We address the following main research question: **How to maximize the asset gains of HFT platforms using an orchestrated multi-model prediction approach?** We analyze the accuracy-latency trade-off in the context of several orchestrator designs, and experimentally evaluate their performance using real-world financial data. Furthermore, we compare the performance of the orchestrated approach against traditional single-

model strategies, and analyze the conditions under which an orchestrator provides a significant advantage.

II. Background

To understand the challenges in predicting market trends in HFT platforms, we first need to understand the structure of financial markets, and current developments in HFT strategies. In this section, we provide an overview of the literature and lay the groundwork for our proposed approach.

A. Market Structure & High-Frequency Data

Traditional quote-driven markets are characterized by a single maker which provides liquidity by quoting both buy and sell prices. In contrast, HFT operates within order-driven markets, where multiple participants can place orders, and the best bid and ask prices are determined by the order book. Event-driven market environments are characterized by a continuous flow of events, usually structured in a Limit Order Book (LOB)—a data structure that organizes buy and sell orders based on price levels and time priority [7]. Unlike quote-driven markets, the LOB maintains a hierarchy of orders, with the best bid and ask prices at the top, and subsequent levels representing deeper liquidity. This hierarchy is known as the *bid-ask spread*, which is the difference between the best ask and best bid prices.

The dynamics of the LOB are influenced by the interaction of three main types of orders: limit orders (or in plain terms, orders to buy or sell at a specified price), market orders (orders to buy or sell immediately at the best available price), and cancellations (withdrawal of previously placed limit orders) [8]. The resulting order flow creates a complex and dynamic stream of events that can be analyzed to predict short-term price movements, a task complicated by *concept drift*—the phenomenon where the statistical properties of the target variable change over time, rendering models trained on historical data less effective in predicting future outcomes.

Capturing the subtle patterns in the LOB data is the key to successful prediction of market trends. The *shelf-life* of orders (i.e., how long they remain in the order book before being executed or canceled) is typically very short, often measured in milliseconds. Additionally, the high-dimensionality of the data and the presence of noise make it challenging to extract meaningful features for prediction [9]. Both of these factors combined contribute to the complexity of modeling HFT data; on one hand, the rapid changes in the order book require models that can adapt quickly to new information, while on the other hand, the high-dimensionality and noise necessitate robust feature extraction techniques. Therefore, we identify the main trade-off in HFT prediction modelling: optimizing for highest possible accuracy while ensuring that the model can adapt to the rapidly changing market conditions fast enough to be useful in real-time trading scenarios.

B. Evolution of Predictive Modeling in Finance

Financial forecasting has evolved significantly over the years, with early approaches relying on linear statistical models such as ARIMA or GARCH. These models assume stable, linear relationships in the data, which often fail to capture the complex, non-linear dynamics of financial markets [1]. The current state-of-the-art in financial modeling has shifted towards architectures with diverse *inductive biases*—the assumptions that a model makes about the data to learn effectively. In the context of HFT, these biases can be optimized to capture unique patterns in the order book data. Intuitively, no single model architecture can capture all the nuances of HFT data.

Tree-based models, specifically *Random forests (RF)* offer a non-sequential baseline [10], or in other words, they can capture complex interactions between features without assuming any temporal structure in the data. RF are used to assess whether the temporal depth of the data is necessary for accurate predictions, or if a simpler model can suffice. *Long short-term memory (LSTM)*-based models, on the other hand, are designed to capture long-term dependencies in sequential data (through a look-back window, which is a hyperparameter that determines how many past steps the model considers when making predictions) [11]. The disadvantage of LSTM models is that they can be computationally expensive and may struggle with very long sequences, which is a common scenario in HFT data. Finally, *Patch Time Series Transformer (PatchTST)* models adapt the transformer architecture to time-series data through two key modifications: (i) the input sequence is divided into patches, which are then processed in parallel, and (ii) the attention mechanism is modified to capture both local and global dependencies in the data [12].

Despite these advancements, no single architecture has emerged as the definitive solution for HFT prediction, due to the bias-variance trade-off and the need for models to adapt to concept drift.

C. Meta-model Orchestration in HFT

The necessity of adapting model orchestration is rooted in the *No Free Lunch* theorem [13], stating that no single predictive model can **consistently outperform** others across diverse datasets and tasks. In HFT, this phenomenon is manifested as *concept drift*; the LOB shifts between periods of high and low volatility. We opt for a Dynamic Model Selection (DMS) approach rather than traditional ensemble averaging. While ensembles aggregate multiple learners to reduce variance, DMS specifically addresses regime-dependent performance by isolating the most capable model for a given market state. In the case of HFT, traditional averaging may introduce unnecessary noise [1], [14].

We analyze the viability of DMS in relation to a *break-even* point—the point at which the performance gain from the meta-model's selection outweighs the inference overhead of maintaining multiple base models and the meta-model itself. Consequently, the design decisions for the meta-model, such as its architecture and the features it uses for selection, are rooted in the delicate balancing act between inference speed and predictive accuracy.

III. METHODOLOGY

In this section, we analyze the design choices and trade-offs involved in an orchestrator-based approach to adaptive market prediction models for HFT applications. Our approach is structured in three stages: (i) the analysis of data and features involved in market prediction models, (ii) the design of orchestrator architectures based on the aggregate performance of a set of baseline models, and (iii) the evaluation of orchestrator performance against the baselines using a comprehensive set of metrics.

Note that this work is not focused on designing a new baseline market prediction model, but rather use existing models as a basis for the orchestrator design and evaluation. To answer the main research question, we focus on the design of a meta-model that can select the best-performing baseline model for a given market regime, based on a set of meta-features that capture both the current market state and the recent performance of the baseline models.

A. Dataset & Feature Space

All experiments use the FI-2010 LOB benchmark [15], a publicly available dataset for high-frequency order book snapshots recorded from the NASDAQ Nordic exchange. FI-2010 covers ten consecutive trading days (1st to 14th of June 2010) of LOB data of five Finnish stocks. The FI-2010 dataset has become the standard benchmark for mid-price prediction in LOB microstructure literature [9], which provides a controlled and reproducible evaluation setting. Each snapshot is a 149-row record, where the first 144 rows encode the LOB feature space (detailed below) and row 148 provides the directional label at prediction horizon $k = 5$. All splits follow the walk-forward evaluation scheme defined by the original benchmark: the first seven days are used for training the base models, day eight serves as the validation set from which orchestrator training labels are derived (see section III-D), and days nine and ten are held aside for final evaluation.

Shuffling is not applied at any stage, which preserves the strictly chronological structure that is required in order to avoid look-ahead bias. The prediction target is a 3-class label $y_t \in \{0, 1, 2\}$ which corresponds to; downward, stationary, and upward mid-price movement over a fixed forward horizon. Following [15], labels are derived from a smoothed forward mid-price to reduce noise from bid-ask oscillations.

The base-model input at event t is a 144-dimensional snapshot $\mathbf{x}_t \in \mathbb{R}^{144}$ made up of two groups of features. The first 40 dimensions encode the raw limit order book state at the top ten price levels on both sides of the book (see eq. (1)), where $p^{\text{ask}} * \ell$ and $p^{\text{bid}} * \ell$ are the ask and bid prices at depth level ℓ, and $v^{\text{ask}} * \ell$, v_ℓ^{bid} are the corresponding (queued) volumes.

$$\left[p_{1,t}^{\text{ask}},\ v_{1,t}^{\text{ask}},\ p_{1,t}^{\text{bid}},\ v_{1,t}^{\text{bid}},\ \ldots,\ p_{10,t}^{\text{ask}},\ v_{10,t}^{\text{ask}},\ p_{10,t}^{\text{bid}},\ v_{10,t}^{\text{bid}}\right] \quad (1)$$

The remaining 104 features are time-insensitive statistics derived from the raw LOB state: inter-level price differences, volume averages, and bid-ask midpoints aggregated across the ten visible levels, as defined by the FI-2010 benchmark protocol [15]. All 144 features are pre-normalised using a z-score transformation computed on a per-day basis following the benchmark convention.

The FI-2010 benchmark was preferred over available raw exchange-feed alternatives because it provides a standardized evaluation protocol that facilitates direct comparison with prior work [9], [15]. The implications of using FI-2010 as a proxy are discussed as a threat to validity in Section VI.

B. Baseline market prediction models

We selected three classifiers with complementary inductive biases as baselines and orchestrator candidates: random forests (non-temporal), long short-term memory (recurrent), and Patch Time Series Transformer (attention-based). All models consume the feature vector $\mathbf{x}_t \in \mathbb{R}^{144}$ described in section III-A and predict a 3-class directional label. The sequential models (LSTM and PatchTST) operate on a sliding window of $L = 100$ consecutive snapshots, producing $M = N - L + 1$ predictions, where N is the total number of snapshots in a given split. The RF produces one prediction per snapshot; the first $L - 1 = 99$ predictions are omitted to align all outputs to the same M-length sequence.

The RF classifies each snapshot independently, testing whether temporal context is necessary for accurate prediction. The ensemble consists of $B = 300$ decision trees, each trained on a bootstrap sample of the training set. Splits maximize the Gini impurity reduction with inverse-frequency class weights $w_c = N_{\text{train}}/(3 \cdot n_c)$, where n_c is the count of class c. Trees are grown to a maximum depth of 20. During the orchestrator's training phase, we use out-of-bag (OOB) predictions rather than in-sample predictions; since each bootstrap sample includes a given instance with probability $1-(1-1/N_{\text{train}})^{N_{\text{train}}} \approx 63.2\%$, the remaining $\approx 36.8\%$ of trees yield unbiased out-of-sample predictions that avoid data leakage into the meta-model.

The LSTM captures path-dependent price dynamics over the L-step window. We stack two LSTM layers ($d = 64$ hidden units each) with dropout ($p = 0.2$) applied between layers. Only the final hidden state $\mathbf{h}_L$ is retained, normalized via layer normalization, and projected to 3-class logits. The model is trained for 30 epochs with Adam [16] (learning rate 1×10^{-3}) and cosine-annealing [17] scheduling, minimizing the same inverse-frequency-weighted cross-entropy loss as the RF. Gradients are clipped to a maximum norm of 1.0.

PatchTST's 100-step input is segmented into patches of length $P = 10$ with stride $S = 5$, yielding $N_p = \lfloor (L - P)/S \rfloor + 1 = 19$ patches per channel. Each of the 144 features is treated as an independent univariate channel (channel-independent design), reducing per-layer attention cost from $O((144 \times 19)^2)$ to $144 \times O(19^2)$. A stack of 3 Transformer encoder layers (4 heads, $d_{\text{model}} = 64$, feed-forward dimension 256, dropout 0.1) processes the patch embeddings. The model leverages transfer learning, initializing from a publicly available checkpoint pretrained on diverse time-series distributions [12] and fine-tuning on the FI-2010 training split for 15

epochs, minimizing unweighted cross-entropy loss (preserving fine-tuning stability, as the near-balanced class distribution makes reweighting unnecessary) with the AdamW optimizer (learning rate 5×10^{-5}) [18] and a batch size of 1024.

C. Evaluation metrics

Our evaluation is structured around two axes that reflect the core trade-off in HFT systems: *prediction quality* and *computational cost*. We assess prediction quality through four complementary metrics that progressively capture finer aspects of classifier behavior, and computational cost through inference latency.

Directional accuracy is the fraction of correctly predicted mid-price directions over all M test events (see Section III-B for the alignment procedure):

$$\text{Acc} = \frac{1}{M} \sum_{t=1}^{M} \mathbf{1}[\hat{y}_t = y_t], \tag{2}$$

where $\mathbf{1}[\cdot]$ is the indicator function, equal to 1 when the predicted class matches the true label and 0 otherwise. While intuitive, accuracy masks performance differences across the three movement classes, so we complement it with more fine-grained metrics.

Macro-averaged F1 [19] computes the harmonic mean of precision and recall per class and averages without weighting:

$$F_1^{\text{macro}} = \frac{1}{C} \sum_{c=1}^{C} F_1^{(c)}, \tag{3}$$

with $C = 3$ (see section A for the per-class precision and recall definitions). The macro-average weights all classes equally, it penalizes a model that neglects any single movement direction, whereas accuracy would mask such a failure in the aggregate count. However, F_1 considers only one class at a time and does not account for true negatives.

Matthews Correlation Coefficient (MCC) [20] addresses this by evaluating all entries of the $C \times C$ confusion matrix $\mathbf{K}$ simultaneously (rather than each class in isolation):

$$\text{MCC} = \frac{n \cdot s - \sum_c p_c t_c}{\sqrt{s^2 - \sum_c p_c^2} \, \sqrt{s^2 - \sum_c t_c^2}}, \tag{4}$$

where $n = \sum_c K_{cc}$, $s = \sum_{i,j} K_{ij}$, $p_c = \sum_j K_{cj}$, and $t_c = \sum_i K_{ic}$. MCC ranges from -1 (complete disagreement) to $+1$ (perfect prediction), with 0 corresponding to random performance, and remains reliable under non-uniform class priors [21]. However, MCC compresses all error types into a single scalar and cannot reveal which specific class confusions drive a low score.

Confusion matrix. To expose per-class error patterns that scalar metrics compress away, we report the full 3×3 matrix. Although the confusion matrix provides the most complete picture of classifier behavior, it does not yield a single rankable number, making direct model comparison less straightforward.

On the computational-cost axis, we report *inference latency*: the per-event wall-clock time (in milliseconds) required to produce a single prediction, averaged over 200 forward passes at batch size 1 after a 10-call warm-up (see section IV-B for hardware details). Because latency is hardware-dependent, we focus on relative comparisons between models.

D. Feature Engineering for Model Orchestration

Rather than operating on raw market data, the orchestrator takes as input a *meta-feature vector* $\mathbf{z}_t$ (see eq. (5)) built from three groups of signals: (i) the market regime, (ii) the base models' predictions, and (iii) the base models' recent performance described as their rolling cross-entropy loss. In this section we describe the rationale behind the selection of these features, and how they are computed from the raw LOB data and the base models' outputs.

$$\mathbf{z}_t = \left[\mathbf{z}_t^{\text{regime}},\ \mathbf{z}_t^{\text{base outputs}},\ \mathbf{z}_t^{\text{rolling loss}}\right]^\top \in \mathbb{R}^9 \tag{5}$$

While the base models consume the full 144-dimensional LOB snapshot, the orchestrator requires a compact set of market regime features to select the most suitable base model for each event. Three microstructure features are computed directly from level-1 LOB columns. The *order-flow imbalance* at time t, OFI_t reflects the difference between best-bid and best-ask volume at the top level of the book. Since all the features are z-score normalized, the volumes can lie close to zero; thus a raw difference is used in preference to the ratio form of [22] to avoid division by near-zero values. The *bid-ask spread*, s_t serves as a direct measure for market liquidity. It is defined as the difference between the best ask and best bid prices, normalized by the mid-price to ensure scale invariance. The *mid-price volatility* $\hat{\sigma}_t$ is computed as the 20-event rolling standard deviation of the mid-price, and used as a market regime indicator. Together, these three features encode the current market regime, $\mathbf{z}_t^{\text{regime}}$ for the orchestrator:

$$\mathbf{z}_t^{\text{regime}} = \left[\text{OFI}_t,\ s_t,\ \hat{\sigma}_t\right]^\top \in \mathbb{R}^3 \tag{6}$$

The basesline models' outputs, $\mathbf{z}_t^{\text{base outputs}}$ are the predicted probabilities of an upward mid-price movement from each of the three base models at time t, where $\hat{y}_t^{(k)}$ is model k's predicted probability.

$$\mathbf{z}_t^{\text{base outputs}} = \left[\hat{y}_t^{(1)},\ \hat{y}_t^{(2)},\ \hat{y}_t^{(3)}\right]^\top \in \mathbb{R}^3 \tag{7}$$

The rolling loss features, $\mathbf{z}_t^{\text{rolling loss}}$ capture the recent performance of each base model, computed as the mean cross-entropy loss over the preceding $\tau = 20$ bars, where $\ell_{t-\tau}^{(k)}$ is the rolling loss for model k at time t.

$$\mathbf{z}_t^{\text{rolling loss}} = \left[\ell_{t-\tau}^{(1)},\ \ell_{t-\tau}^{(2)},\ \ell_{t-\tau}^{(3)}\right]^\top \in \mathbb{R}^3, \tag{8}$$

To generate training labels, all three base models are run in a forward pass over the validation split, keeping the orchestrator's training data separate from the base models'. The ground-truth label k_t^*, hereafter referred to as the *oracle label*, is defined as the base model with the lowest rolling prediction error over the preceding τ bars:

$$k_t^* = \arg\min_k \frac{1}{\tau} \sum_{i=0}^{\tau-1} \mathbf{1}\left[\hat{y}_{t-i}^{(k)} \neq y_{t-i}\right] \tag{9}$$

This frames orchestration as a 3-class classification task, trained with categorical cross-entropy, where $\hat{g}_t^{(k)}$ is the gating network's softmax weight for model k. Class imbalance is corrected by inverse-frequency weighting, missing values are filled by linear interpolation, and all features in $\mathbf{z}_t$ are only standardized on the training split.

$$\mathcal{L}_{\text{orch}} = -\frac{1}{T} \sum_{t=1}^{T} \sum_{k=1}^{3} \mathbf{1}[k_t^* = k] \ \log \hat{g}_t^{(k)} \tag{10}$$

E. Orchestrator Meta-model Architecture

Recall the goal of the orchestrator in regard to the main research question: to select the best-performing baseline model for a given market regime, based on a set of meta-features (see eq. (5)) that capture both the current market state and the recent performance of the baseline models. We highlight the need to minimize inference overhead in HFT applications, imposing a lightweight design constraint on the orchestrator architecture. Therefore, the orchestrator should function as a lightweight meta-model, that takes the meta-features $\mathbf{z}_t$ as input and outputs a discrete selection of the best-performing baseline model for time t. We choose to implement two orchestrator architectures: (i) a multinomial logistic regression model, and (ii) a multi-layer perceptron. We refer to these as the *logistic orchestrator* and the *MLP orchestrator* respectively.

The logistic orchestrator serves as a linear baseline, with the advantage of interpretability and minimal computational overhead. It models the probability of selecting model k via a softmax transformation of the linear response. Equation (11) shows the logistic orchestrator's output, where $\mathbf{w}_k$ and b_k are the weights and bias for model k, and σ is the softmax function. The model outputs the probability that the oracle label k_t^* corresponds to model k, given the meta-features $\mathbf{z}_t$. To make a decision at time t, the orchestrator selects the model with the highest predicted probability as the best-performing model for that time step.

$$P(k_t^* = k \mid \mathbf{z}_t) = \sigma(\mathbf{w}_k^\top \mathbf{z}_t + b_k) \tag{11}$$

The MLP orchestrator introduces non-linearity to capture more complex relationships between the meta-features and the best-performing model. The feed-forward multi-layer perceptron consists of two hidden layers of 32 and 16 units respectively. We use *Rectified Linear Unit (ReLU)* activation function for the hidden layers (see eqs. (12) and (13)) to introduce non-linearity, while maintaining computational efficiency. ReLU is defined as $\text{ReLU}(x) = \max(0, x)$, which allows the model to learn non-linear relationships without the vanishing gradient problem [23] associated with other activation functions. The model is trained using weighted cross-entropy loss, specifically $\mathcal{L}_{\text{orch}}$ as defined in eq. (10). Equation (14) shows the output layer of the MLP orchestrator, which produces a vector of logits that are then transformed into probabilities via a softmax function, similar to the logistic orchestrator.

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1 \mathbf{z}_t + \mathbf{b}_1) \tag{12}$$

$$\mathbf{h}_2 = \text{ReLU}(\mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2) \tag{13}$$

$$\mathbf{o}_t = \mathbf{W}_3 \mathbf{h}_2 + \mathbf{b}_3 \tag{14}$$

We evaluate both orchestrator architectures against the baseline models in section V, comparing their performance in terms of predictive accuracy and computational efficiency.

F. Orchestrator evaluation & trade-offs

While the orchestrator is designed to improve prediction accuracy, it does so at the cost of adding a decision layer on top of three already-running models. This means that any accuracy gain must be weighed against the extra inference time it introduces. This leads to the **break-even** concept—the point at which the latency overhead cancels out the predictive advantage provided by this novel approach. The hypothesis is that the MLP orchestrator is expected to achieve higher accuracy than LR, but at the cost of increased latency. The orchestrator is assumed to show the largest gains over individual baselines in high-volatility regimes, where the best model switches frequently whereas in stable, low-volatility regimes the overhead may not be justified, and a single model may suffice. Additionally, it is believed that RF alone may be competitive in latency-critical settings despite lower accuracy. This being due to RF running on CPU without a sliding window, which means it classifies a single 144-dimensional snapshot independently, with no sequence processing overhead. While LSTM and PatchTST both require a 100-step context window to be assembled and passed through sequential or attention layers before a prediction is made, which takes more time. In HFT settings where every millisecond matters, RF's simplicity makes it the fastest option even if it sacrifices some accuracy in predictions.

Taking these points into consideration, there are a few boundaries to establish. The signal's shelf-life indicates that if the market moves before the orchestrated prediction is acted upon, any accuracy gain becomes economically irrelevant. This implies that in practice, the orchestrator is only worthwhile if the accuracy gain, scaled by average trade value, exceeds the latency cost scaled by missed opportunities. Building on this, specific metrics are established in line with the above considerations. The ratio of per-model inference time to total orchestrator pipeline time is set to quantify the latency overhead introduced by the gating layer. Additionally, the accuracy and macro F1 delta between the best single baseline and the orchestrated system directly measures whether the added complexity yields a meaningful predictive improvement. Finally, a breakdown of which model the orchestrator selects across different market conditions reveals whether the gating network learns meaningful regime-dependent behavior, or if it defaults to a single model regardless of context.

IV. Experimental Setup

In this section, we outline the empirical setup used to evaluate the proposed HFT meta-model orchestrator. We connect the theoretical base of section III to practical evaluation, by detailing the temporal partitioning of data, the training protocol, and the evaluation strategy.

A. Temporal Data Partitioning

To maintain the HFT environment's temporal integrity and prevent look-ahead bias, we adhere to a strictly chronological walk-forward partition of the ten-day FI-2010 dataset, as described in section III-A. The first stage is training, partitioned on days one through seven, which are used as the initial parameter window for the three baseline models (see section III-B). In the second stage–validation, we use day 8 to generate out-of-sample predictions and build the oracle labels k_t^*, and finally the meta-feature vectors $\mathbf{z}_t$ for the orchestrator training. Finally, the last two days are held as the testing set for the final evaluation of the orchestrator's performance against the baselines.

B. Inference Benchmarking

We quantify the computation cost by profiling the inference time of the orchestrator and the three baseline models on the test set. For reproducibility purposes, it is relevant to mention the hardware specifications of the machine used for inference benchmarking, which is equipped with an AMD EPYC 9375F CPU (32 physical cores, 64 threads), 256GB DDR5 RAM, and an NVIDIA L4 GPU with 24GB GDDR6 memory.

Each benchmark consists of a warm-up phase of 10 iterations, followed by 200 timed runs, measuring the average inference time per event for each model in milliseconds. For orchestrated models, the total latency is defined as the sum of the inferences of all the three baseline models plus the decision-making overhead of the gating model, which is the orchestrator's inference time.

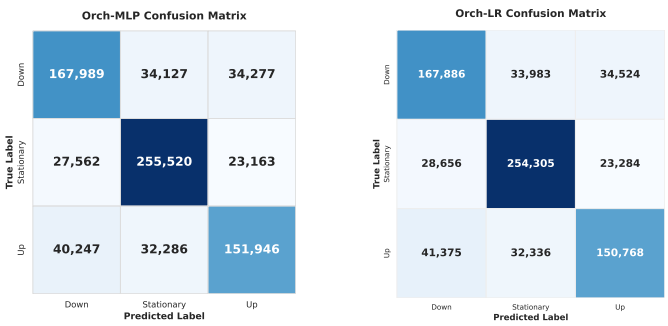
C. Measurements

We report the following performance metrics for the orchestrator and the three baseline models on the test set, according to the defined metrics in section III-C:

1) Predictive power: we report the directional accuracy, Macro-F1, and Matthews Correlation Coefficient for all models, and compare the orchestrator's performance against the baselines.
2) Model behavior: we analyze the 3×3 confusion matrices for each model (baseline and orchestrator) to understand the types of errors made, and how the orchestrator's decision-making affects the distribution of predictions across the three classes defined in section III-A.
3) Efficiency: we analyze the absolute latency in milliseconds and the relative overhead ratio of the orchestrator compared to the best-performing baseline model, to understand the trade-off between predictive performance and computational cost in the HFT context.

V. Results

In this section, we present the results of our experiments, comparing the performance of the orchestrated models against the baseline models, according to section IV-C.



(a) MLP orchestrator. (b) Logistic orchestrator.

Fig. 1: Confusion matrices for the two orchestrated models.

Figures 1 and 2 indicate that the orchestrated models, particularly the MLP-based orchestrator, achieve a more balanced performance across the three classes (Down, Stationary, Up), with fewer misclassifications compared to the baseline models. This result is consistent with our hypothesis that the meta-model orchestrator can leverage the strengths of each baseline model to improve overall predictive performance, especially in the more challenging classes (Down and Up), which are often underrepresented in the dataset.

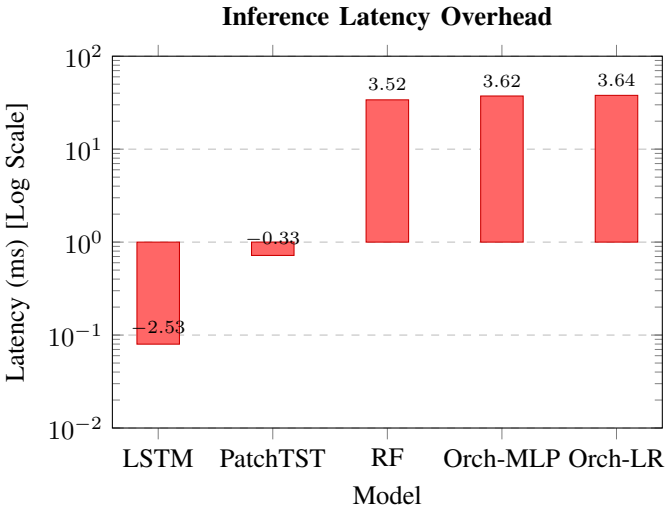


Fig. 4: Inference latency per model (logarithmic scale). The orchestration pipeline introduces a significant computational overhead ($\approx$ 37 ms) compared to standalone sequential models like the LSTM (0.08 ms).

The evaluation on the FI-2010 dataset show that sequential deep learning models outperform traditional static algorithms (see fig. 3). The proposed non-linear MLP orchestrator performs comparably to the LSTM baseline, achieving a 75.01% accuracy and an MCC of 0.62. Both models successfully

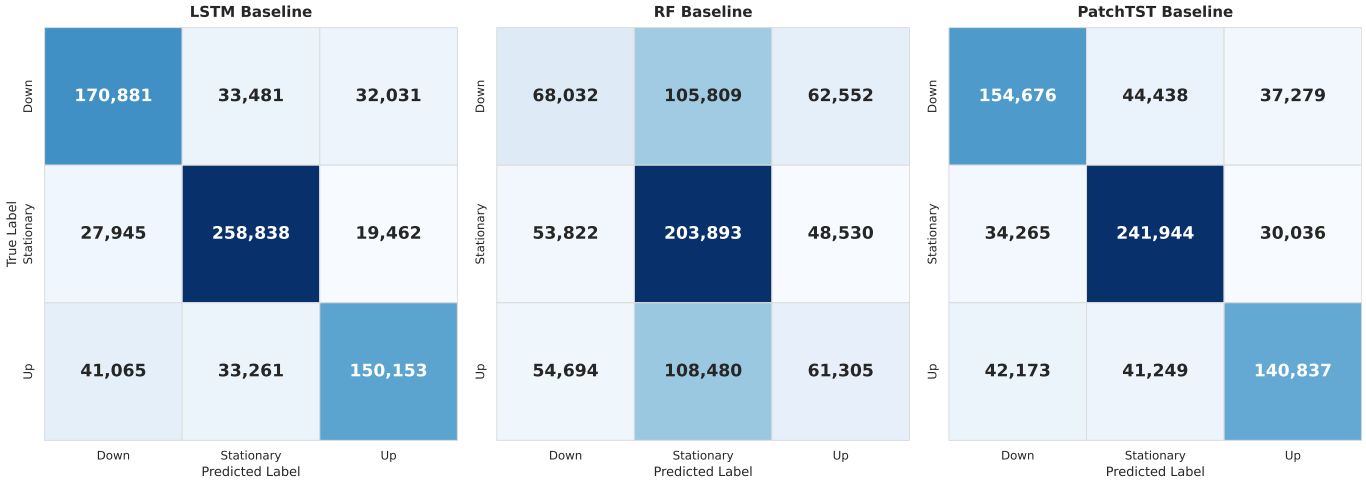


Fig. 2: Side-by-side comparison of baseline models' confusion matrices.

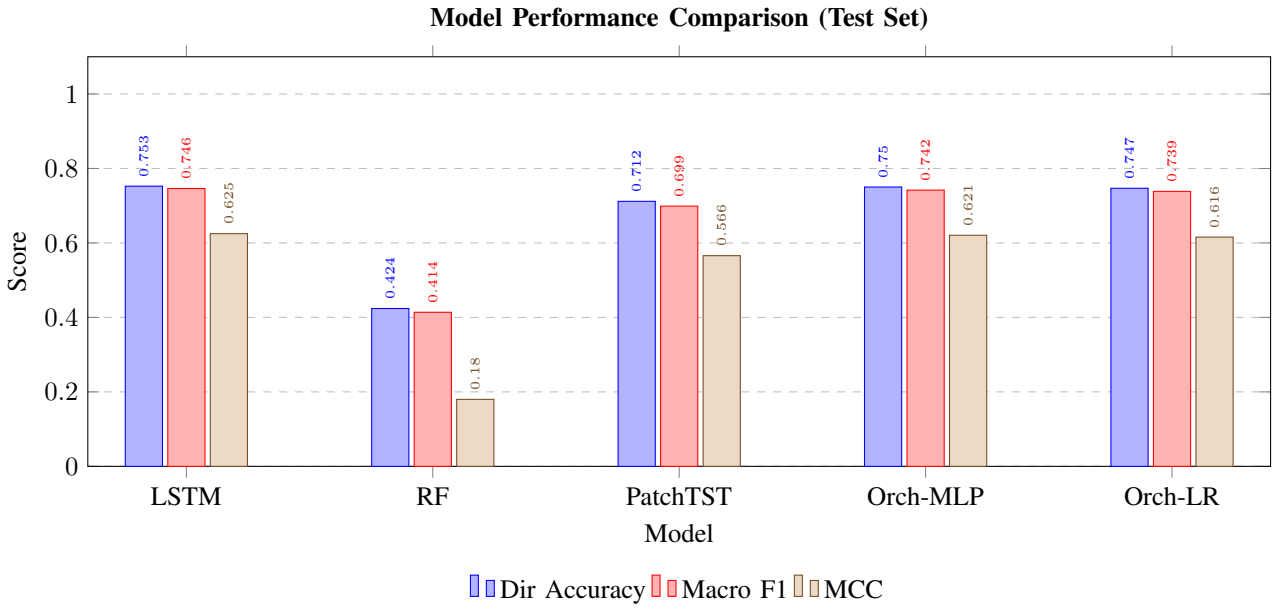


Fig. 3: Predictive metrics across baseline and orchestrated models. The score is computed according to eqs. (2) to (4) respectively. Note that the orchestrated models perform comparably to the LSTM baseline, and outperforming the RF and PatchTST baselines.

captured the temporary dynamics of the LOB, but RF failed to generalize, yielding an accuracy of only 42.39%. PatchTST performed moderately, with an accuracy of 71.17%, suggesting that attention mechanisms are not as well suited for the immediate, tick-level features of the LOB data, compared to the LSTM's ability to capture temporal dependencies.

The central limitation of the orchestrator is its computational overhead in regard to its *break-even point*, as described in section III-F. Figure 4 shows that the inference latency for a single forward pass of both orchestrator models is approximately 37 ms, orders of magnitude higher than the 0.08 ms latency of the LSTM baseline. This significant overhead is primarily due to the need to run all three baseline models for each prediction.

VI. Discussion & Result Analysis

We have demonstrated that the proposed orchestrated model, particularly the MLP-based orchestrator, achieves a more balanced performance across the three classes (Down, Stationary, Up) compared to the baseline models. Notably, the oracle label distribution reveals that the LSTM is selected as the best-performing model in approximately 84% of cases, suggesting that the complementary-bias hypothesis is only weakly supported: the orchestrator largely learns to defer to a single model. Although promising, the orchestrator's performance is not without limitations, particularly in terms of inference latency, which is significantly higher than that of the LSTM baseline. In this section, we discuss the implications of our

results for real-world applications, the potential benefits and challenges of incorporating sentiment analysis as an additional input feature group, and the limitations of our current approach.

A. Viability of Orchestrated Models in Real-World HFT Applications

The results of our experiments suggest that while orchestrated models can improve predictive performance by leveraging the strengths of multiple baseline models, their practical viability in real-world HFT applications is limited by the slowest inference of the baseline models. In our case, the RF baseline bottlenecked the orchestrator's latency, making it unsuitable for high-frequency trading. Consequently, the standalone LSTM baseline is more suitable for real-world deployment, as its inference latency is orders of magnitude lower than that of the orchestrated models, while still achieving competitive performance.

B. Sentiment Analysis as an Additional Input Feature Group

The existing meta-feature vector z_t only captures contemporaneous market state but is inherently reactive; signals like $\hat{\sigma}_t$ and OFI_t react to price moves after they occur. Incorporating a FinBERT-derived sentiment layer would extend z_t with two signals: a polarity score $\psi_t^{\text{sent}} \in [-1, 1]$ aggregated over a rolling news window, and its first difference ρ_t^{sent} capturing narrative momentum - the trend direction. The primary expected benefit is reduced switching lag at regime boundaries: currently the gating network only shifts weight toward RNN-based baselines only when volatility has already spiked, but a leading sentiment signal could anticipate transitions driven by earnings releases or macro announcements before they hit the price.

In practice, this extension proposes two constraints. First, latency: FinBERT is too slow to run inline in an HFT setting, and would require asynchronous scoring with a shared cache to avoid slowing down predictions past the point where they're still useful. Second, signal quality: outside of major news events, sentiment adds mostly noise, and ideally, the gating network's learned weights for ψ_t^{sent} should converge toward zero in such conditions to avoid degrading baseline performance. A further limitation is that the oracle labeling scheme of Equation (6) rewards retrospective accuracy rather than early correct switching, meaning the training objective would need modification to fully exploit the anticipatory value of sentiment features.

C. Limitations

Several limitations should be acknowledged. The FI-2010 dataset is from 2010 and covers only five Finnish equities from the NASDAQ Nordic exchange. Markets are inherently volatile, and data from more than a decade ago will hardly generalize to the behavior of modern markets or different exchanges. Furthermore, the oracle labeling scheme (see eq. (9)) selects the best-performing models based on retrospective accuracies. It rewards models that *were accurate*, instead of those that *will be*. This causes a loss of performance on regime shifts. Third, all hyperparameters of the models were chosen manually, instead of a systematic search. Potential gains were left unexplored. Finally, the models were evaluated on a single prediction horizon ($k = 5$) and a single train-test partition; increasing the horizon and making repeated splits would potentially make the outputs more generalizable.

VII. Future Work

The focus of this paper is on predicting mid-price direction on a single LOB benchmark through a meta-learning orchestrator that selects the best model per market regime. While this approach provides a rigorous and well-scoped foundation for financial market research, predicting direction is only half the problem; how much to trade on each signal matters enormously for actual profit and loss. Therefore, as a natural extension of this work, future research could incorporate position sizing and risk management strategies that translate directional signals into concrete trading decisions. In addition, given that financial markets are heavily influenced by news and general sentiment, integrating FinBERT-derived sentiment signals as leading regime indicators — as discussed in section VI, provided that the latency and asynchronous scoring challenges are resolved first, represents a promising direction.

VIII. Conclusion

This research is motivated by a deeper exploration of the *No Free Lunch* problem in HFT. We proposed a DMS orchestrator that routes predictions across RF, LSTM, and PatchTST based on a 9-dimensional meta-feature vector capturing market regime, model confidence, and rolling loss. The orchestrated approach does not outperform the best single baseline in terms of the accuracy-latency trade-off.

The study demonstrated that the MLP orchestrator matches LSTM on accuracy (75.01% versus 75.3%) however at $\tilde{3}7$ms versus 0.08ms the break-even point is never reached. The accuracy gap between RF (42%) and and the temporal models (LSTM/PatchTST $> 71\%$) conclusively shows that instantaneous LOB snapshots are not sufficient for mid-price prediction. Instead, it is the sequential order flow dynamics that carry the predictive signal. The results reveal that the latency bottleneck is not inherent to the gating model itself, but rather to the requirement of running all three base models for every prediction, with RF specifically bottlenecking the pipeline. As seen in the results, MLP marginally edges LR across all metrics (0.750 vs. 0.747 accuracy; 0.621 vs. 0.616 MCC) at near-identical latency, suggesting that the non-linear capacity of the MLP provides minimal benefit for this meta-feature space. The primary limitations of this work are the use of a single benchmark dataset (FI-2010), the restriction to one market, and hardware-dependent latency measurements that are not directly transferable.

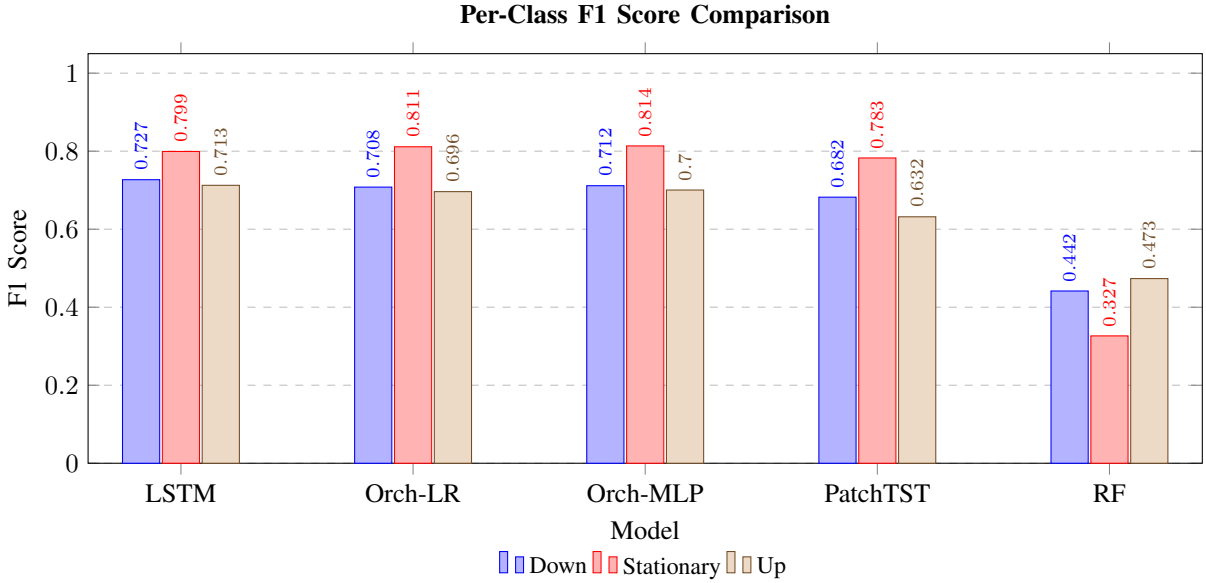


Fig. 5: Breakdown of F1 scores across the three directional classes for baseline and orchestrated models.

References

[1] S. Gu, B. Kelly, and D. Xiu, "Empirical asset pricing via machine learning," *The Review of Financial Studies*, vol. 33, no. 5, pp. 2223–2273, 2020.

[2] R. Abdelghani, "Evaluation of high-frequency prediction approaches in price determination on financial markets," *European Public & Social Innovation Review*, vol. 10, pp. 1–15, 01 2025.

[3] K. Batool, M. M. Baig, and U. Fatima, "Meta-learning for financial market prediction: An efficient approach with reduced computational cost," *International Journal of Advanced and Applied Sciences*, vol. 12, no. 11, pp. 72–81, 2025.

[4] K. Noor and U. Fatima, "Meta learning strategies for comparative and efficient adaptation for financial datasets," *IEEE Access / ResearchGate Preprint*, vol. 13, p. 24159, 2025. [Online]. Available: https://www.researchgate.net/publication/388914837

[5] R. Contributors, "Adaptive market intelligence: A mixture of experts framework for volatility-sensitive stock forecasting," *ResearchGate / International Journal of Forecasting*, 2025. [Online]. Available: https://www.researchgate.net/publication/394322530

[6] A. W. Lo, "The adaptive market hypothesis," *Journal of Portfolio Management*, vol. 30, no. 5, pp. 15–29, 2004.

[7] R. Cont, "A functional central limit theorem for limit order books," *SIAM Journal on Financial Mathematics*, vol. 1, no. 1, pp. 588–607, 2010.

[8] M. D. Gould, M. A. Porter, S. Williams, M. McDonald, D. J. Fenn, and S. D. Howison, "Limit order books," *Quantitative Finance*, vol. 13, no. 11, pp. 1709–1742, 2013.

[9] Z. Zhang, S. Zohren, and S. Roberts, "Deeplob: Deep convolutional neural networks for limit order books," *IEEE Transactions on Signal Processing*, vol. 67, no. 11, pp. 3001–3012, 2019.

[10] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[11] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[12] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *International Conference on Learning Representations (ICLR)*, 2023.

[13] D. H. Wolpert, "The lack of a priori distinctions between learning algorithms," *Neural Computation*, vol. 8, no. 7, pp. 1341–1390, 1996.

[14] A. G. Rossi and S. P. Utkus, "Predicting stock market returns with aggregate data: A model selection approach," *Review of Financial Studies*, 2018.

[15] A. Ntakaris, M. Magris, J. Kanniainen, M. Gabbouj, and A. Iosifidis, "Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods," *Journal of Forecasting*, vol. 37, no. 8, pp. 852–866, 2018.

[16] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2017. [Online]. Available: https://arxiv.org/abs/1412.6980

[17] I. Loshchilov and F. Hutter, "Sgdr: Stochastic gradient descent with warm restarts," 2017. [Online]. Available: https://arxiv.org/abs/1608.03983

[18] ——, "Decoupled weight decay regularization," 2019. [Online]. Available: https://arxiv.org/abs/1711.05101

[19] C. J. V. Rijsbergen, *Information Retrieval*, 2nd ed. Butterworth-Heinemann, 1979.

[20] J. Gorodkin, "Comparing two k-category assignments by a k-category correlation coefficient," *Computational Biology and Chemistry*, vol. 28, no. 5, pp. 367–374, 2004. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1476927104000799

[21] D. Chicco and G. Jurman, "The advantages of the matthews correlation coefficient (mcc) over f1 score and accuracy in binary classification evaluation," *BMC Genomics*, vol. 21, 01 2020.

[22] R. Cont, A. Kukanov, and S. Stoikov, "The price impact of order book events," *Journal of Financial Economics*, vol. 112, no. 1, pp. 47–88, 2014.

[23] X. Glorot, A. Bordes, and Y. Bengio, "Deep sparse rectifier neural networks," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, 2011, pp. 315–323.

Appendix

The per-class $F_1^{(c)}$ used in eq. (3) is the harmonic mean of precision and recall:

$$F_1^{(c)} = \frac{2\,\mathrm{Pr}^{(c)} \cdot \mathrm{Re}^{(c)}}{\mathrm{Pr}^{(c)} + \mathrm{Re}^{(c)}}, \tag{15}$$

where

$$\mathrm{Pr}^{(c)} = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c}, \qquad \mathrm{Re}^{(c)} = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FN}_c}, \tag{16}$$

and TP_c, FP_c, FN_c are the true positives, false positives, and false negatives for class $c \in \{0, 1, 2\}$ respectively. When both $\mathrm{Pr}^{(c)}$ and $\mathrm{Re}^{(c)}$ are zero, $F_1^{(c)}$ is defined as 0 by convention.